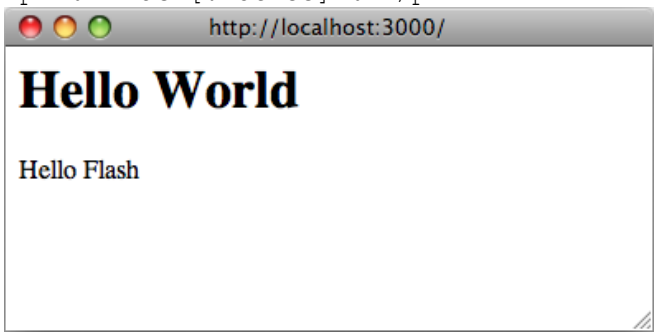
OK! Now you've initialized I18n support for your Ruby on Rails application and told it which locale to use and how to preserve it between requests. With that in place, you're now ready for the really interesting stuff.

Let's *internationalize* our application, i.e. abstract every locale-specific parts, and then *localize* it, i.e. provide necessary translations for these abstracts.
You most probably have something like this in one of your applications:

```
# config/routes.rb
Rails.application.routes.draw do
  root to: "home#index"
end
# app/controllers/application_controller.rb
class ApplicationController < ActionController::Base
  before_action :set_locale

  def set_locale
    I18n.locale = params[:locale] || I18n.default_locale
  end
end
# app/controllers/home_controller.rb
class HomeController < ApplicationController
  def index
    flash[:notice] = "Hello Flash"
  end
end
# app/views/home/index.html.erb
<h1>Hello World</h1>
<p><%= flash[:notice] %></p>
```



3.1 Adding Translations

Obviously there are **two strings that are localized to English**. In order to internationalize this code,**replace these strings** with calls to Rails' `#t` helper with a key that makes sense for the translation:

```
# app/controllers/home_controller.rb
class HomeController < ApplicationController
  def index
    flash[:notice] = t(:hello_flash)
  end
end
# app/views/home/index.html.erb
<h1><%=t :hello_world %></h1>
<p><%= flash[:notice] %></p>
```

When you now render this view, it will show an error message which tells you that the translations for the keys `:hello_world` and `:hello_flash` are missing.